ELLMo: Packing- and Depth-Aware Encrypted Transformer Inference

Seyda Nur Guzelhan
Boston University
seyda@bu.edu

Lohit Daksha
Boston University
tihol@bu.edu

Carlos Agulló Domingo
University of Murcia
carlos.a.d@um.es

Gilbert Jonatan
KAIST
gilbertjonatan@kaist.ac.kr

John Kim
KAIST
jjk12@kaist.ac.kr

José L. Abellán
University of Murcia
jlabellan@um.es

David Kaeli
Northeastern University
kaeli@ece.neu.edu

Ajay Joshi
Boston University
joshi@bu.edu

***Abstract*—Cloud-based Large Language Model (LLM) inference processes sensitive user inputs, yet current deployments offer limited confidentiality guarantees. Fully Homomorphic Encryption (FHE) can provide strong privacy, but it clashes with transformer architectures, where rigid ciphertext packing demands expensive rotations, and deep polynomial circuits for nonlinearities necessitate costly bootstrapping. Although recent work has reported promising speedups, maintaining model accuracy is a challenge. We address these issues with ELLMo, a packing- and depth-aware encrypted transformer design. ELLMo introduces a novel matrix multiplication algorithm to reduce ciphertext rotations. Further, head-split and merge steps are fused into this new algorithm at no additional cost. To reduce the depth of nonlinear layers, our contributions, Statistical-$\max$ Softmax and DelayNorm, help bypass deep comparison trees and homomorphic divisions to reduce bootstrapping by up to 46%. On encrypted BERT-Tiny, ELLMo achieves a $1.4\times$ speedup over state-of-the-art baselines with 0%–1.5% accuracy loss across SST-2, MRPC, and RTE downstream tasks.**

1. Introduction

Enterprises deploy Large Language Models (LLMs) in the cloud to leverage scalable compute resources. They send queries involving medical history, financial records, or proprietary business intelligence to the cloud for processing. However, cloud-based inference involves processing these sensitive input queries in plaintext, raising critical privacy concerns. Fully Homomorphic Encryption (FHE) addresses this by enabling the cloud to process data while it remains encrypted [26]. When using FHE, the user first encrypts the input and then the LLM operates on ciphertexts without decryption. Only the user can decrypt the final result [28].

FHE promises end-to-end confidentiality, but, unfortunately, it is not practical as it introduces large computational and memory overheads. To mitigate this, on the hardware front, researchers have developed specialized accelerators [1], [4], [37], [58]. From an algorithmic perspective, several studies have proposed to optimize data packing and homomorphic matrix multiplication for more efficient encrypted linear layers [47], [50], [67]. Finally, tailored polynomial approximations can be implemented to process transformer nonlinearities more efficiently under FHE [15], [69]. Still, encrypted inference remains prohibitively costly, often lagging orders of magnitude behind plaintext execution [21]. The root cause is not merely raw operational cost, but a fundamental structural mismatch between modern transformer architectures and constraints of FHE.

In the encrypted domain, packing allows encrypting multiple values into a single ciphertext to increase throughput and reduce memory overhead. However, this creates a rigid structure. To multiply two matrices, one needs to access the same index of a row (left matrix) and column (right matrix). Data reordering and indexing in the encrypted domain requires an expensive operation called ciphertext rotation. Standard transformer mechanisms exacerbate this problem with constant data reshaping (e.g.,multi-head attention (MHA) splits the data into heads, and feed-forward networks (FFNs) expand and shrink the data size).

Cheon-Kim-Kim-Song (CKKS) has emerged as the FHE scheme of choice for privacy-preserving ML, because it enables high throughput linear transformations on real numbers [12]. On the other hand, it approximates non-linear functions via polynomial expansions. A degree-n polynomial induces a multiplication chain of depth $\log n$ [51]. High-degree polynomial approximations quickly exhaust the noise budget, forcing expensive bootstrapping operations. This poses several challenges, because representing LLM nonlinearities at high precision results in frequent bootstrapping calls. Softmax and LayerNorm require a division operation, and GELU behavior is critical near zero. Additionally, Softmax involves a $\max$ subtraction step to control the growth of exponentials, and computing the $\max$ over T inputs results in a comparison circuit of depth $\log T$. Some existing implementations choose to use the $\max$ value obtained from the training data [17] or a portion of the input sequence [50]; however, these may violate the confidentiality guarantees required to protect sensitive user information in the cloud. This creates a tension: current systems must choose between high bootstrapping costs with confidentiality guarantees or keep the bootstrapping cost low, but compromise on privacy and accuracy.

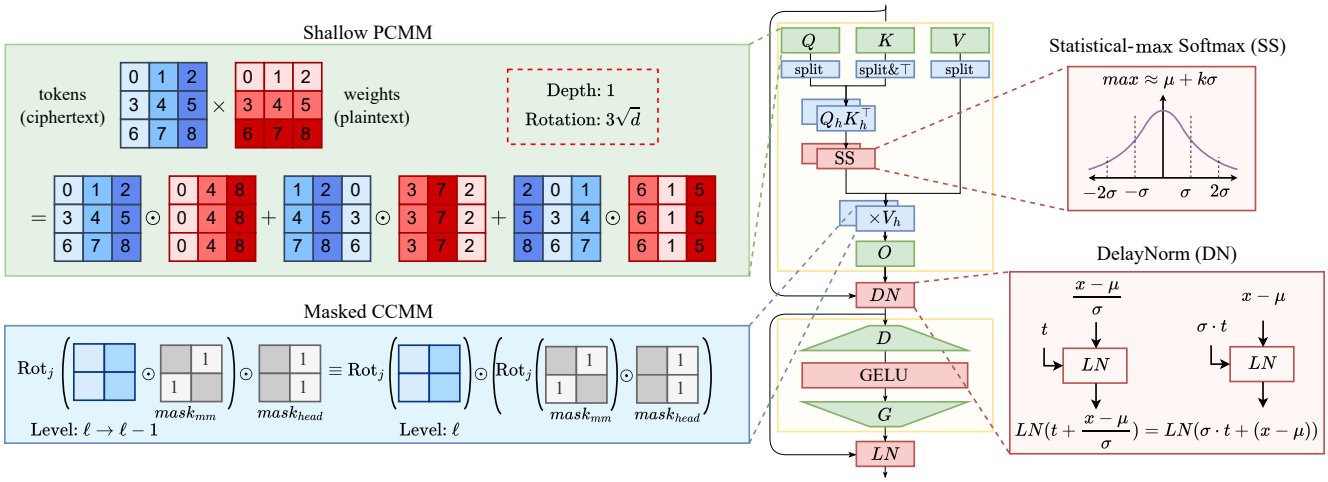


Figure 1: **ELLMo: FHE-friendly Transformer** *(Top Left)* Shallow PCMM reduces rotation costs and levels consumed. *(Bottom Left)* Masked CCMM fuses head-splitting into existing transforms to amortize the cost. *(Top Right)* Statistical-max Softmax replaces the depth-expensive comparison tree with a statistical upper bound, ensuring stability without leaking training data statistics. *(Bottom Right)* DelayNorm avoids immediate normalization by propagating the scaling factor until the next LayerNorm.

To address these challenges, we propose ELLMo, a privacy-preserving inference system built on packing- and depth-aware innovations. Rather than only optimizing low-level operations, we redesign how the model itself performs computation. Figure 1 summarizes ELLMo's contributions.

To reduce the cost of linear layers, ELLMo optimizes encrypted matrix multiplications with packing-aware techniques. First, ELLMo proposes a novel matrix multiplication algorithm by two key observations: 1) **Cost asymmetry:** Operations on encrypted data are an order of magnitude more expensive than the same operations on plaintext [3]. We consider the scenario where transformer inputs are encrypted, while weights are plaintext. If we skip some portion of the rotations on ciphertexts, we can perform them on plaintext weights at a lower cost. 2) **Pre-encoding Transforms:** Plaintext transforms can be applied prior to encoding, only requiring cheap floating point arithmetic on real vectors. Based on these intuitions, we skip the traditional two-step JKLS schedules in the state-of-the-art implementations, and implement the matrix multiplication with only one multiplicative level. We call our method shallow PCMM, and reduce the runtime by 30% while halving the multiplicative depth. Second, ELLMo fuses MHA split and merge steps to its matrix multiplication algorithm. Standard implementations isolate each attention head via a separate masking step, which increases the multiplicative depth [54]. By fusing head isolation into matrix multiplication, we eliminate the additional depth consumption.

Third, ELLMo develops privacy-preserving polynomial approximations that remove dependence on plaintext statistics. For Softmax, we approximate the maximum using Bernstein-type bounds on sub-exponential distributions [8], replacing plaintext-derived normalization with a statistical approximation to max. Further, ELLMo also packs all attention heads jointly, enabling a single-pass Softmax instead of redundant per-head approximations.

Finally, ELLMo introduces a scale-invariant optimization for normalization in encrypted inference. LayerNorm computes mean-centered outputs divided by their standard deviation and is invariant to uniform input scaling. ELLMo exploits this by replacing homomorphic division with cheaper multiplications: rather than dividing by the standard deviation, we let the scaling propagate through subsequent linear layers. The following LayerNorm cancels this shared scale by construction, preserving correctness while eliminating the polynomial approximation of division.

To summarize ELLMo's contributions:

- **Packing-Aware Transformer (PAT):** A shallow JKLS-based PCMM that shifts rotations from ciphertexts to plaintext, reducing key-switching by up to 50% and depth from two levels to one; and a fused-masking CCMM that integrates head-splitting into the linear transforms, reducing masked CCMM depth from four to three levels.
- **Statistical-max Softmax (SS):** A statistical max estimation for numeric stability, combined with a one-shot, jointly packed multi-head evaluation to avoid redundant polynomial calls.
- **DelayNorm (DN):** A scale-invariant LayerNorm variant that replaces homomorphic division with cheap multiplications by deferring normalization: the scale factor is propagated through subsequent linear layers and canceled by the next LayerNorm.

We evaluate ELLMo in both plaintext and encrypted settings on the GLUE downstream tasks Stanford Sentiment Treebank-2 (SST-2) [48], Microsoft Research Paraphrase Corpus (MRPC) [19], and Recognizing Textual Entailment (RTE) [27]. In the plaintext domain, we run BERT-Tiny [36]

and BERT-Base [18] with plaintext models and inputs, comparing the original architectures to variants that use ELLMo's DN and SS (PAT only applies to encrypted execution). For BERT-Base, DN causes only a 0.10% accuracy drop on SST-2 and a 0.004 F1 decrease on MRPC, mirroring the small losses for BERT-Tiny and showing that ELLMo's LayerNorm modification preserves accuracy even for larger models. In the encrypted domain, we implement ELLMo for CKKS-encrypted BERT-Tiny [49], [54], [56] and evaluate with all three contributions enabled (PAT+SS+DN) on NVIDIA H200 GPUs, using the open-source FIDESlib implementation [4] as our baseline. ELLMo achieves an average $1.4\times$ speedup over FIDESlib, with no accuracy loss on SST-2, a 1% drop on MRPC, and a 1.5% drop on RTE.

2. Preliminaries

In this section, we present the background needed for ELLMo. We first describe the threat model, then review the CKKS homomorphic encryption (HE) scheme and its core primitives. We next explain how matrix multiplications and nonlinear functions are mapped to the encrypted domain, highlighting the resulting efficiency bottlenecks. Finally, we summarize the transformer architecture that serves as our target encrypted workload. Table 1 summarizes the CKKS cryptographic parameters and transformer model notation used in this work.

2.1. Threat Model

We assume a setting where user inputs are encrypted with CKKS and processed by a cloud service that holds plaintext model weights. The cloud is semi-honest (honest-but-curious): it follows the prescribed computation but may attempt to infer information from the ciphertexts. Under this model, user-data privacy is guaranteed by the semantic security of CKKS, while model parameters remain unencrypted and visible to the provider. This matches common outsourced inference scenarios where clients protect sensitive queries (e.g., medical or financial text) while leveraging large pretrained models hosted by an untrusted cloud.

2.2. CKKS Background

Many FHE schemes have been proposed, including TFHE and FHEW for Boolean circuits [14], [20], BGV and BFV for integer arithmetic [9], [23], and CKKS for approximate real-valued computation [12]. The CKKS scheme is designed for approximate arithmetic on real (or complex) numbers and allows for the packing of multiple plaintext values in a single ciphertext [12]. Secure instantiations of CKKS with bootstrapping require a ring dimension on the order of 2^{16}–2^{17} and ciphertext moduli exceeding one thousand bits [7]. To implement these large moduli efficiently, CKKS adopts the Residue Number System (RNS) decomposition [11], which leverages the Chinese Remainder Theorem to split the modulus into machine-word-sized primes. Therefore, we employ the RNS variant of CKKS [10], which is the standard for efficient encrypted computation.

We summarize the CKKS-RNS primitives below [3]:

- **KeyGen(λ):** Given a target security level λ, sample secret key s, random polynomial a, and small error e. Set public key $pk = (a, b)$ with $b \approx -a \cdot s + e$. Generate linearization key $evk_{s^2 \to s}$ and rotation keys $evk_{\Phi_j(s) \to s}$.
- **Encode(μ, Δ):** Apply inverse Fast-Fourier Transform (IFFT) on a vector $\mu \in \mathbb{R}^n$. Scale the result by Δ and round to the nearest integer to get a plaintext polynomial $m \in \mathbb{Z}(x)/(x^N + 1)$.
- **Encrypt(m, pk):** Sample u, e_0, e_1 from the noise distributions and output $c = (c_0, c_1) = ((-a \cdot s + e) \cdot u + m + e_0, a \cdot u + e_1)$.
- **Decrypt(c, s):** Decrypt ciphertext c into an approximate plaintext $c_0 + s \cdot c_1 = m' \approx m$.
- **Add(c, c'):** Produce $c_{add} \approx \text{Encrypt}(m + m')$ from $c = \text{Encrypt}(m)$ and $c' = \text{Encrypt}(m')$.
- **Mult($c, c', evk_{s^2 \to s}$):** Compute the raw product

$$(d_0, d_1, d_2) = (c_0 c_0',\ c_0 c_1' + c_1 c_0',\ c_1 c_1')$$

  then relinearize d_2 to $(\tilde{d}_0, \tilde{d}_1)$ using key-switching with $evk_{s^2 \to s}$. As the scales multiply, apply a Rescale to control growth.
- **PtRot$_j$(m):** Apply Φ_j (cyclic slot shift by j) to plaintext m (all indices are modulo n):

$$\Phi_j(m) = \{m_j, \ldots, m_{n-1}, m_0, \ldots, m_{j-1}\} \quad (1)$$

- **Rot$_j$($c, evk_{\Phi_j(s) \to s}$):** Apply automorphism Φ_j, then key-switch back to s.

$$\text{Rot}_j(c) = \text{Key-Switch}(\Phi_j(c), evk_{\Phi_j(s) \to s}) \quad (2)$$

- **Key-Switch($c, evk_{s_1 \to s_2}$):** Using a switching key from s_1 to s_2, transform c encrypting m under s_1 into $\tilde{c}$ decryptable under s_2, preserving the scale and level up to a small noise increase. Special cases: (i) relinearization with $evk_{s^2 \to s}$ after multiplication to remove the s^2 term; (ii) rotation with $evk_{\Phi_j(s) \to s}$ after applying Φ_j to undo the secret-key permutation.
- **Rescale(c, Δ_ℓ):** Divide the scale by Δ_ℓ by dropping one RNS modulus, reducing remaining multiplicative depth.

When levels are exhausted, Bootstrap refreshes the accumulated noise via homomorphic linear transforms and approximate modular reduction, restoring the available level budget to L_{eff} (with $L_{\text{eff}} > \ell$). Since bootstrapping is composed of numerous rotations and multiplications, it is computationally expensive.

2.3. Homomorphic Matrix Multiplication

Matrix multiplication in the encrypted domain relies heavily on ciphertext rotations to align rows and columns, and each rotation permutes the underlying secret key and therefore requires a key-switching operation. Key-switching

Table 1: **Notation summary.** CKKS parameters and transformer notation used throughout the paper.

Symbol	Description
N	Polynomial ring degree.
n	Number of slots.
QP	Ciphertext modulus.
λ	Target security level in bits.
L	Maximum multiplicative depth.
ℓ	Current level of a ciphertext.
L_{eff}	Available depth after bootstrapping.
$\times$	Matrix multiplication.
$\pi, \tau, \phi, \psi, \zeta$	Linear transforms in encrypted matrix multiplication.
$\odot$	Element-wise (Hadamard) product.
T	Input sequence length (tokens).
d	Hidden dimension.
H	Number of attention heads.
d_h	Per-head width, $d_h = d/H$.
$W_{Q,\ K,\ V,\ O}$	MHA projection weights in $\mathbb{R}^{d\times d}$.
$W_{D,\ G}$	FFN projection weights in $\mathbb{R}^{d\times 4d}$ and $\mathbb{R}^{4d\times d}$.
μ, σ	Mean and standard deviation of a distribution.
γ, β	Learned LayerNorm parameters in $\mathbb{R}^d$.

is the most expensive CKKS primitive [4], [68], [71], so the number of rotations (and hence key-switch calls) largely dominates the cost of homomorphic matrix multiplication.

Jiang-Kim-Lauter-Song (JKLS) Method. The JKLS method [34] is a structured approach that reduces the number of ciphertext rotations by reorganizing operands via linear transforms. Let A and B be $d \times d$ matrices, where A is in ciphertext and B can be in plaintext (for PCMM) or ciphertext (for CCMM). Instead of aligning every row of A with every column of B individually, JKLS defines permutations that let all products be computed in parallel by element-wise multiplications. On input matrices $A, B \in \mathbb{R}^{d\times d}$, JKLS uses the following linear transforms: $\pi(A)_{i,j} = A_{i,\,i+j}$, $\tau(B)_{i,j} = B_{i+j,\,j}$, $\phi^k(A)_{i,j} = A_{i,\,j+k}$, and $\psi^k(B)_{i,j} = B_{i+k,\,j}$, all indices taken modulo d [34]. A homomorphic linear transform t on a matrix X is computed as:

$$t(X) = \sum_{j=0}^{d-1} (\mathrm{Rot}_j(X) \odot M_{t_j}) \tag{3}$$

where M_t are the plaintext masks to isolate the entries based on the indexing requirements of transform t.

Concretely, let π and τ transform a 3×3 matrix X with ordered entries $\{1, \cdots, 9\}$. π packs the upper diagonals of X in its columns, while τ packs the lower diagonals of X in its rows:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}; \begin{array}{l} \text{Upper diagonals: } [1,5,9],[2,6,7],[3,4,8] \\ \text{Lower diagonals: } [1,5,9],[4,8,3],[7,2,6] \end{array}$$

$$\pi(X) = \begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 4 \\ 9 & 7 & 8 \end{bmatrix}; \quad \tau(X) = \begin{bmatrix} 1 & 5 & 9 \\ 4 & 8 & 3 \\ 7 & 2 & 6 \end{bmatrix} \tag{4}$$

Let ϕ^k and ψ^k denote the row and column reorderings, where ϕ^1 rotates 1 column and ψ^1 rotates 1 row as:

$$\phi^1(\pi(X)) = \begin{bmatrix} 2 & 3 & 1 \\ 6 & 4 & 5 \\ 7 & 8 & 9 \end{bmatrix}; \quad \psi^1(\tau(X)) = \begin{bmatrix} 4 & 8 & 3 \\ 7 & 2 & 6 \\ 1 & 5 & 9 \end{bmatrix} \tag{5}$$

With all these four transforms, the matrix product is computed by:

$$A \times B \ = \ \sum_{k=0}^{d-1} \left(\phi^k \circ \pi(A)\right) \ \odot \ \left(\psi^k \circ \tau(B)\right) \text{ [34]} \tag{6}$$

where $\circ$ denotes function composition and $\odot$ denotes element-wise multiplication.

In CKKS, JKLS requires two multiplicative levels for PCMM and three for CCMM, i.e., it performs two (or three) sequential ciphertext multiplications, each followed by a $\mathrm{Rescale}$ that consumes one level. These linear transforms form the basis for ELLMo's optimizations: in Section 3, we propose a shallow PCMM variant that shifts one transform from A to B, exploiting the fact that plaintext operations are an order of magnitude cheaper than ciphertext operations [4], and a masked CCMM variant that fuses MHA masking directly into the JKLS transforms to avoid additional level consumption.

Baby-Step Giant-Step (BSGS) Algorithm. To further reduce rotations, we apply the baby-step giant-step algorithm [30]. For a sum over d rotations, BSGS reduces the key-switch operations from $O(d)$ to $O(\sqrt{d})$ by decomposing the rotation index into 'b' baby steps and 'g' giant steps, chosen as powers of 2 near $\sqrt{d}$ (e.g., $b = 2^{\lfloor \log_2 \sqrt{d} \rfloor}$ and $g = 2^{\lceil \log_2 \sqrt{d} \rceil}$). This decomposition reuses intermediate rotations and significantly lowers the total number of key-switch calls. JKLS naturally integrates with BSGS by structuring its rotate-and-sum loops to exploit this decomposition.

2.4. Nonlinear Function Approximation

Transformer nonlinearities include the maximum, exponentials, and reciprocals (for Softmax); the Gaussian error function (GELU); the inverse standard deviation (LayerNorm); and $\tanh$ in classification heads. Each is challenging under CKKS: exponentials grow rapidly, the maximum over a length-T vector needs an $O(\log T)$ comparison tree, reciprocals are singular at zero, the inverse standard deviation requires both square root and inversion, GELU depends on the Gaussian error function, and $\tanh$ saturates quickly outside a narrow interval. These difficulties motivate efficient polynomial approximation techniques.

For bounded intervals, Chebyshev expansions yield minimax (maximum-error minimizing) polynomial approximations [46]. When higher precision is needed for reciprocals, we refine an initial polynomial estimate with Newton–Raphson iterations, at the cost of additional levels [17], [52]. In Sections 3 and 4, we build on this foundation and develop ELLMo's specialized approximations for encrypted transformers that reduce depth to minimize bootstrapping.

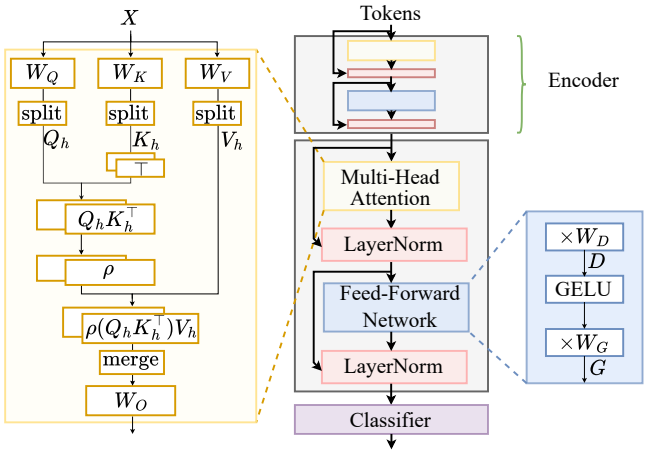


Figure 2: **BERT-Tiny network architecture.** The model consists of two encoder layers followed by a classifier. Each encoder contains a multi-head attention block and a feed-forward network (FFN), both with residual connections and LayerNorm. The attention block employs query, key, value, and output projections (Q, K, V, O) and applies Softmax activation ρ. The FFN uses two linear projections (D, G) with a GELU activation in between.

2.5. Transformers

Transformer-based LLMs stack identical encoder layers that convert token embeddings into contextual representations [62]. Each layer comprises a multi-head attention (MHA) and a feed-forward network (FFN), wrapped with residual connections and LayerNorm (LN). Below we explain the basics of the transformer model architecture using BERT-Tiny. Figure 2 shows the BERT-Tiny model architecture with two encoders, followed by a classification head [36].

MHA involves linear projections Q, K, V, O; a nonlinear activation Softmax; and applies split and merge steps to operate on H heads in parallel. The input token matrix has size $T \times d$, where T and d are the number of tokens and hidden dimension, respectively. Then, the Q, K, V projections of input tokens X are:

$$Q = X \times W_Q, \qquad K = X \times W_K, \qquad V = X \times W_V. \tag{7}$$

The split step partitions each matrix into H heads as

$$Q_h \;=\; Q\{:,\, h\, d_h : (h+1)\, d_h\}, \qquad h \in [H] \tag{8}$$

where $(:)$ chooses all rows. Split applies to Q, K and V.

Softmax (ρ) is computed row-wise on the attention scores $Q_h \times {K_h}^\top$ per head:

$$\rho\left(\frac{Q_h \times {K_h}^\top}{\sqrt{d_h}}\right) \tag{9}$$

where the factor $1/\sqrt{d_h}$ stabilizes the variance of the logits.

Then, MHA merges the H heads and applies another linear projection O. LayerNorm (LN) shifts and scales the MHA output as

$$\mathrm{LN}(x) \;=\; \gamma\, \frac{x-\mu}{\sigma} + \beta \tag{10}$$

where μ and σ are the mean and standard deviation per token, and γ and β are learnable parameters.

The FFN applies two linear maps with an intermediate GELU nonlinearity. This subblock, similar to MHA, is wrapped with residual connections and LN. After passing through the stack of MHA and FFN layers (2 layers in BERT-Tiny), the final output is sent to a classifier to make a prediction (e.g., "Is this email spam or not?").

3. Efficient Homomorphic Matrix Multiplications

Despite significant progress in FHE-based language models, severe performance bottlenecks remain: ciphertext rotations dominate runtime due to costly key-switching, and multi-head attention (MHA) adds overhead because head splitting and merging rely on masking that increases multiplicative depth. ELLMo addresses these challenges with two optimizations: (i) a shallow PCMM that shifts rotations from ciphertext to plaintext, reducing key-switching calls and lowering depth from two to one; and (ii) a masked CCMM that integrates head masking into the linear transforms, eliminating extra level consumption.

3.1. Shallow PCMM

Let A and B be $d \times d$ matrices, stored in ciphertext and plaintext, respectively. Recall from Section 2 that JKLS-type matrix multiplication follows three steps: Step 1 applies a π transform on A, Step 2 applies τ on B, and Step 3 computes the product via ϕ on A and ψ on B, followed by an element-wise multiplication. Recent variants improve upon this template: THOR [47] proposes a new diagonal packing to reduce ciphertext rotations, while BLB [67] reorders Step 3 to deploy the Baby-Step-Giant-Step (BSGS) across transforms. These optimizations reduce costs, yet the bottleneck of an excessive number of key-switching remains.

The standard JKLS-based PCMM methods, while effective at reducing rotations compared to naive approaches, still treat both operands symmetrically, applying analogous linear transforms to each matrix regardless of their computational costs. However, in the plaintext-ciphertext multiplication setting, this symmetry is fundamentally misaligned with the underlying cost model. We now present our shallow PCMM approach that exploits this cost asymmetry.

In PCMM, operations on ciphertext matrix A require expensive key-switching after each rotation (Rot), while operations on plaintext matrix B use only plaintext rotations (PtRot) that require no key-switching. Since Rot is approximately an order of magnitude more expensive than PtRot [3], [4], we can improve efficiency by shifting computational burden from the ciphertext to the plaintext.

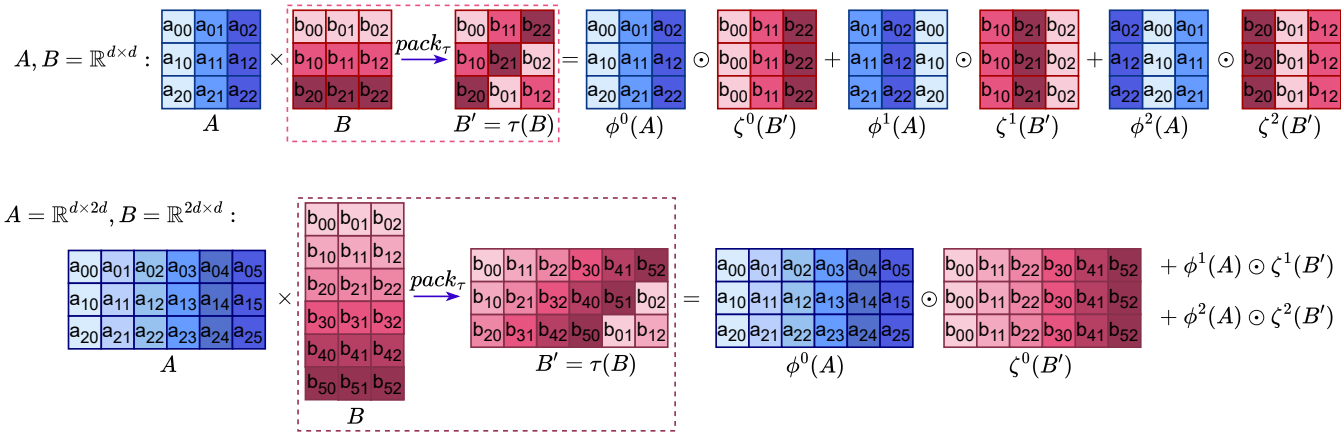


Figure 3: **Shallow plaintext-ciphertext matrix multiplication (PCMM).** *Top:* ELLMo's PCMM algorithm eliminates the π transform on the ciphertext matrix A via replacing the ψ transform on B with ζ. It further eliminates the τ transform on plaintext B by applying the transform during the packing step, denoted as $pack_\tau$. The resulting algorithm requires one transform on each input, consuming only one level. *Bottom:* Rectangular ELLMo shallow PCMM. As long as the plaintext B packed with $pack_\tau$, ELLMo shallow PCMM generalizes to any $A = \mathbb{R}^{d_1\times d_2}$, $B = \mathbb{R}^{d_2\times d_1}$, where d_1 and d_2 are powers of 2 for efficient packing. The figure illustrates the case $d_2 = 2d_1$.

ELLMo's approach: We eliminate the π transform on the A entirely. Recall from Section 2.3, the matrix product $A \times B$ can be computed through d element-wise products between different permutations of $\psi \circ \tau(B)$ and $\phi \circ \pi(A)$. This requires two transforms on the plaintext and ciphertext matrices each. We define a new *row-select* transform ζ to replace ψ. ζ defines a fundamentally different transformation: Unlike ψ which reorders the rows of $\tau(B)$, ζ selects only the k-th row and replicates it across the matrix as:

$$\tau(X) = \begin{bmatrix} 1 & 5 & 9 \\ 4 & 8 & 3 \\ 7 & 2 & 6 \end{bmatrix}; \quad \zeta^0(\tau(X)) = \begin{bmatrix} 1 & 5 & 9 \\ 1 & 5 & 9 \\ 1 & 5 & 9 \end{bmatrix} \tag{11}$$

which corresponds to

$$\zeta^k(B)_{i,j} = B_{k,j} \tag{12}$$

Critically, ζ enables alignment with $\phi(A)$, without needing to apply π on A first. Thus, ζ correctly calculates the matrix product, while avoiding the π transform on A entirely.

Proposition 3.1. Correctness of Shallow PCMM:

$$A \times B = \sum_{k=0}^{d-1} \left(\phi^k(A)\right) \odot \left(\zeta^k \circ \tau(B)\right) \tag{13}$$

where $\odot$ denotes the element-wise product.

Proof. Fix $i, j \in \{0, \ldots, d-1\}$. The (i, j) entry of the right-hand side in Equation (13) is

$$\begin{aligned} \sum_{k=0}^{d-1} \phi^k(A)_{i,j} \odot \left(\zeta^k \circ \tau(B)\right)_{i,j} &= \sum_{k=0}^{d-1} A_{i,\,j+k}\ \tau(B)_{k,\,j} \\ &= \sum_{k=0}^{d-1} A_{i,\,j+k}\ B_{j+k,\,j} \end{aligned} \tag{14}$$

As k ranges over $\{0, \ldots, d-1\}$, the index $t = j+k \pmod d$ ranges over all $\{0, \ldots, d-1\}$ exactly once, so

$$\sum_{k=0}^{d-1} A_{i,\,j+k}\ B_{j+k,\,j} = \sum_{t=0}^{d-1} A_{i,t}\ B_{t,j} = (A \times B)_{i,j}. \tag{15}$$

Since this holds for all (i, j), the matrices are equal. □

Second, we eliminate Step-2 by the following observation: Pre-computing $\tau(B)$ and encoding the result is equivalent to encoding B and applying the homomorphic linear transformation τ. Since weights are plaintext, we can skip Step 1-2 by pre-computing $\tau(B)$ prior to encoding, which we denote as $pack_\tau$. This results in omitting one transform on A and B each, reducing the multiplicative depth to one. Hence, we call our method shallow PCMM. Algorithm 1 summarizes shallow PCMM.

Cost of shallow PCMM. We implement the linear transforms using BSGS: for a length-d transform we choose

$$b = 2^{\lfloor \log_2 \sqrt{d} \rfloor}, \qquad g = 2^{\lceil \log_2 \sqrt{d} \rceil}. \tag{16}$$

and perform b baby-step rotations and g giant steps. With our optimization, after the b baby-step rotations we accumulate ciphertexts in a loop with giant step g and rotate only the running sum, avoiding $(g-1)$ rotations on each input. As a result, shallow PCMM uses

$$K_{\text{shallow}}(d) \;=\; 2(b-1) + g \approx 3\sqrt{d} \tag{17}$$

key-switching operations for a PCMM of size $d \times d$.

The original JKLS paper implements the transform with a cost on the order of $O(N)$. However, for a fair comparison, we consider an optimized implementation of the same algorithm with $O(\log N)$ complexity [67]. In the (optimized) baseline JKLS, the π transform of length-$2d$ has a cost of $K_\pi(2d) \;=\; (b'-1) + g' \approx 3\sqrt{d}$ key-switchings, where b' and g' are computed for $2d$ per Equation 16. Thus, the

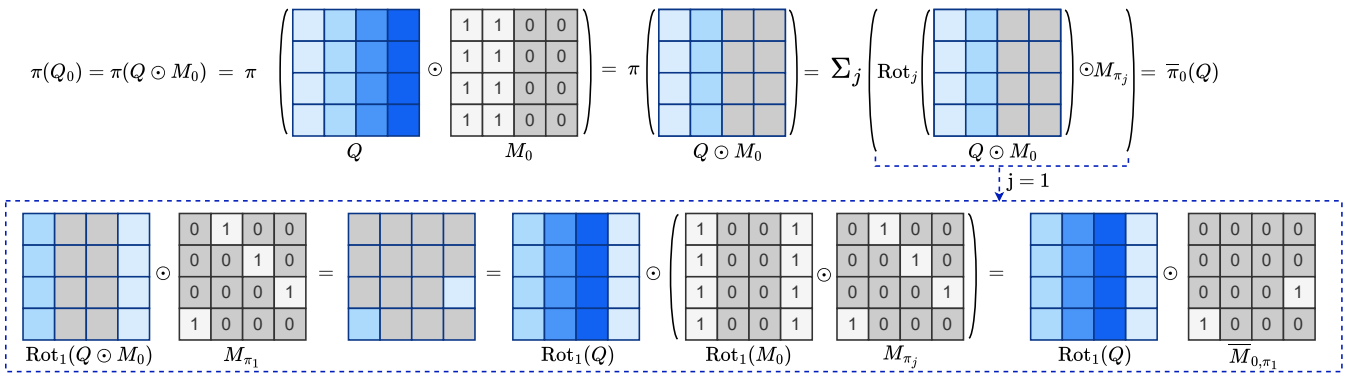


Figure 4: **Masked ciphertext–ciphertext matrix multiplication (CCMM).** *Top:* The equivalence $\pi(Q_0) = \overline{\pi}_0(Q)$ for level-efficient CCMM. In standard multi-head attention, extracting head $Q_0 = Q \odot M_0$ costs one level, and the naive CCMM $\tau(Q_0)$ applies transform masks M_{π_j}, costing another. ELLMo's optimized $\overline{\pi}_0(Q)$ fuses the head and transform masks into $M_h \odot M_{\pi_j}$, reducing these two levels to one. The remaining CCMM steps use two levels, for a total depth of 3 versus 4 for the naive implementation. *Bottom:* Mask fusion follows from $\mathrm{Rot}_1(Q \odot M_0) \odot M_{\pi_1} = \mathrm{Rot}_1 Q \odot (M_0 \odot M_{\pi_1}) = \mathrm{Rot}_1 Q \odot \overline{M}_{0,\pi_1}$, eliminating one ciphertext multiplication and level.

Algorithm 1 Shallow PCMM

Require: Ciphertext A, plaintext $B' = \mathrm{Encode}(\tau(B))$
Ensure: Ciphertext AB

$AB \leftarrow 0$
for $k = 0$ **to** $d - 1$ **do**
  $A_k \leftarrow \phi^k(A)$
  $B_k \leftarrow \zeta^k(B')$
  $AB \leftarrow AB + \mathrm{Mult}(A_k, B_k)$
end for
return AB

baseline PCMM uses $K_{\mathrm{base}}(d) = K_{\mathrm{shallow}}(d) + K_\pi(2d) \approx 6\sqrt{d}$, so eliminating π removes half of the rotation calls for PCMM.

Rectangular Shallow PCMM. Rectangular PCMM is a useful extension for models with heterogeneous dimensions across the architecture, such as the transformers. ELLMo's shallow PCMM generalizes beyond square settings with minimal adjustment. The key-switching cost of a rectangular shallow PCMM of size $T \times d$ with $T < d$ is

$$K_{\mathrm{rect,shallow}}(T, d) = 2(b-1) + g + \log_2 s - 1, \quad (18)$$

with $b = 2^{\lfloor \log_2 \sqrt{T} \rfloor}$, $g = 2^{\lceil \log_2 \sqrt{T} \rceil}$, and $s = \lceil d \,/\, T \rceil$. This matches the cost of a square shallow PCMM of size $T \times T$, up to $s-1$ additional rotations for the final summation over $s \cdot T$ columns. Figure 3 presents a simple example with 3×3 matrices for shallow PCMM (top) and its rectangular variant (bottom).

Key Takeaway: ELLMo's shallow PCMM reduces key-switching calls by skipping diagonal transforms and defining a new column-ordering transform. The multiplicative depth reduces from two to one.

3.2. Masked CCMM

Multi-head attention requires a split step to mask each input and isolate heads. In CKKS, masking via PtMult consumes one level. We observe that JKLS linear transformations already involve masking within their rotate-and-sum schedules. We propose fusing the split-step masking with CCMM linear transforms, thereby saving one level. We denote the two ciphertext matrices as Q_h and $K_h^\top$, mirroring the MHA CCMM from Section 2.3. Our algorithm generalizes to any ciphertext matrix pair.

Baseline JKLS CCMM begins with a π transform on $d \times d$ matrix Q using plaintext masks M_{π_j}:

$$\pi(Q) = \sum_{j=-d}^{d-1} \big(\mathrm{Rot}_j(Q) \odot M_{\pi_j}\big) \quad (19)$$

In MHA CCMM, the split step masks the input Q to extract Q_h as:

$$Q_h = M_h \odot Q, \quad h \in [H] \quad (20)$$

using plaintext masks M_h. Then, Q_h undergoes a π transform:

$$\pi(Q_h) = \sum_{j=-d}^{d-1} \big(\mathrm{Rot}_j(M_h \odot Q) \odot M_{\pi_j}\big) \quad (21)$$

Key Observation: Both the JKLS transforms and head splitting require multiplying by a plaintext mask. We apply the following property to avoid duplicate masking:

Lemma 3.1. *For any ciphertext X, rotation index j, and plaintext mask M:*

$$\mathrm{Rot}_j\big(M \odot X\big) = \mathrm{PtRot}_j(M) \odot \mathrm{Rot}_j(X) \quad (22)$$

Proof. In CKKS (and similar RLWE-based schemes), rotation is a ring automorphism on the cyclotomic polynomial ring [12]. For any ring elements a, b and automorphism Φ: $\Phi(a \cdot b) = \Phi(a) \cdot \Phi(b)$. □

Algorithm 2 Masked CCMM

Require: Ciphertexts Q and $K^\top$, head mask M_h, linear transform masks M_{π_j} where $j \in \{-d, \ldots, d-1\}$.
Ensure: Ciphertext $Q_h\, K_h^\top = (M_h \odot Q) \times K$

$\overline{\pi}(Q) \leftarrow \sum_{j=-d}^{d-1} \left(\mathrm{Rot}_j(Q) \odot \left(\mathrm{PtRot}_j(M_h) \odot M_{\pi_j}\right)\right)$
$Q_h\, K_h^\top \leftarrow 0$
for $k = 0$ **to** $d-1$ **do**
  $Q_h\, K_h^\top \leftarrow Q_h\, K_h^\top \;+\; \mathrm{Mult}(\Phi^k(\overline{\pi}(Q)),\ \psi^k(\tau(K)))$
end for
return $Q_h\, K_h^\top$

We define a new transform $\overline{\pi}_h$ using fused plaintext masks:

Definition 3.1.

$$\overline{\pi}_h(Q) \;=\; \sum_{j=-d}^{d-1} \left(\mathrm{Rot}_j(Q) \;\odot\; \overline{M}_{h,\pi_j}\right) \tag{23}$$

where $\overline{M}_{h,\pi_j} = \mathrm{PtRot}_j(M_h) \;\odot\; M_{\pi_j}$.

Proposition 3.2. (Correctness of Masked CCMM Step-1)

$$\overline{\pi}_h(Q) = \pi(Q_h) \tag{24}$$

Proof. Using Definition 3.1, we have

$$\begin{aligned}\overline{\pi}_h(Q) &= \sum_{j=-d}^{d-1} \left(\mathrm{Rot}_j\left(Q\right) \;\odot\; \overline{M}_{h,\pi_j}\right) \\ &= \sum_{j=-d}^{d-1} \left(\mathrm{Rot}_j\left(Q\right) \;\odot\; \left(\mathrm{PtRot}_j\left(M_h\right) \odot\; M_{\pi_j}\right)\right)\end{aligned}$$

Reordering the elementwise multiplications:

$$= \sum_{j=-d}^{d-1} \left(\left(\mathrm{PtRot}_j(M_h) \odot \mathrm{Rot}_j\left(Q\right)\right) \odot M_{\pi_j}\right)$$

Applying Lemma 3.1:

$$= \sum_{j=-d}^{d-1} \left(\mathrm{Rot}_j\left(M_h \odot Q\right) \odot M_{\pi_j}\right) = \pi(Q_h) \tag{25}$$

which concludes the proof. □

Figure 4 presents a toy example with 4×4 matrices and a Rot_1 applied to masks. This integrates head splitting into CCMM Step 1 with no extra ciphertext–plaintext multiplications, completed in a single level. CCMM continues with Step 2 (applying τ on $K^\top$), and Step 3 (merging via ϕ and ψ transforms). Because only π changes, Proposition 3.2 suffices for correctness. Step 2's linear transform τ has the same form as Equation (19), so the analysis extends to the $K^\top$ input. Generally, any CCMM input Q (or $K^\top$) can defer masking to Step 1 (or 2) to eliminate level consumption.

Algorithm 2 computes the masked CCMM for multi-head attention while fusing the head-splitting mask directly into the JKLS linear transform.

Broader applicability: Although our focus is transformer-based language models, our techniques extend to other ML algorithms. Shallow PCMM accelerates encrypted linear algebra workloads, such as matrix multiplications in logistic regression. Masked CCMM naturally applies to settings requiring selective slot access (e.g., masked convolutions in privacy-preserving image classification).

Key Takeaway: ELLMo's masked CCMM fuses head-masking into existing transforms to amortize the cost.

4. Secure and Efficient Nonlinear Activations

Nonlinear activations are a major bottleneck in homomorphic transformers, often dominating encrypted LLM runtime [4], [17], [54]. ELLMo proposes FHE-friendly alternatives that cut this cost while preserving privacy. For Softmax, ELLMo uses a statistical-$\max$ estimator that stabilizes exponentiation without explicitly computing the maximum. For LayerNorm, ELLMo exploits scale invariance to avoid homomorphic division while retaining its effect. Together, these techniques reduce multiplicative depth and bootstrapping overhead.

4.1. Softmax with Statistical $\max$ Estimation

Recall from Section 2.5 that Softmax is applied row-wise over the sequence length T to the scaled dot-product scores in each attention head. As we compute Softmax independently for each row and head, we omit these subscripts for brevity and denote the attention scores as $\mathbf{s} = \{s_0, \ldots, s_{T-1}\}$. Softmax is then:

$$\rho(\mathbf{s})_i = \frac{e^{s_i}}{\sum_{j=0}^{T-1} e^{s_j}}, \quad i \in [T]. \tag{26}$$

The numerical stability problem. When the input range is large, exponentials can vanish or explode. Standard plaintext practices subtract the maximum value before exponentiation. Let $\alpha = \max_j \mathbf{s}$. Defining $\tilde{s}_i = s_i - \alpha$ guarantees $\tilde{s}_i \le 0$ for all i, with the largest entry at 0. Crucially, this shift preserves the Softmax output:

$$\rho(\mathbf{s} - \alpha)_i = \frac{e^{s_i - \alpha}}{\sum_{j=0}^{T-1} e^{s_j - \alpha}} = \rho(\mathbf{s})_i \tag{27}$$

for any constant α. Using $\alpha = \max_j \mathbf{s}$ avoids overflow and is essential for numerical stability in unencrypted Softmax.

The encrypted dilemma. Computing $\max_j \mathbf{s}$ on ciphertexts is impractical. This is because computing the maximum requires comparing all vector entries: an encrypted comparison reduces to approximating a sign function via high-degree polynomials [45]. Finding the maximum of T values necessitates a comparison circuit of depth $O(\log T)$. For typical sequence lengths (T in [128, 768]), this quickly

exhausts the multiplicative depth budget and forces multiple costly bootstrapping operations. Moreover, using any plaintext-derived maximum violates privacy.

ELLMo's approach: statistical-$\max$ estimation. We estimate the maximum using cheaply computed statistics: the mean μ and standard deviation σ of the logits. Subtracting $\mu + p\,\sigma$ for an appropriate constant p shifts most values below zero without requiring comparisons. The effectiveness of this approach depends on how much probability mass lies beyond p standard deviations. Empirical studies show that attention logits exhibit sub-exponential behavior [31], [65]. Their tails decay slower than Gaussian, but concentration bounds still apply [8].

Under standard attention analyses [62], for an arbitrary head h and token i, a logit is a scaled dot product

$$\frac{1}{\sqrt{d_h}} \sum_{\ell=0}^{d_h-1} Q_h[i,\ell] \cdot K_h^\top[\ell, j], \tag{28}$$

with mean $\mu = 0$ and variance $\sigma^2 = 1$ when row i of Q_h and K_h are independent, zero-mean, unit-variance vectors. Each product term is sub-exponential [63], so the dot product inherits heavier-than-Gaussian tails, consistent with reported measurements of neural representations [31], [65]. We model attention scores using a Laplace distribution, which captures these heavier tails while admitting closed-form coverage expressions. For a Laplace-distributed random variable X with standard deviation σ:

$$\Pr\left(|X| \le p\,\sigma\right) \approx 1 - e^{-\sqrt{2}\,p} \tag{29}$$

To translate this into the fraction of logits that are shifted below zero (and thus safe from overflow), we convert to one-sided coverage:

$$\mathrm{DistCov}(p) = 0.5 + \frac{1}{2}\Pr\left(|X| \le p\,\sigma\right) \approx 1 - \frac{1}{2}e^{-\sqrt{2}\,p} \tag{30}$$

This matches intuition: $\mathrm{DistCov}(0) = 0.5$ (shifting by the mean covers half the mass), and $\mathrm{DistCov}(p) \to 1$ as p increases. Setting $p \in [1, 2]$ provides $85 - 95\%$ coverage, sufficient to prevent overflow while avoiding exact $\max$ computation. This provides a statistically grounded, privacy-preserving approximation sufficient for numerical stability.

Head-wise Merging for One-Pass Softmax: We also optimize the parallel processing of Softmax across attention heads. Softmax operates on each head individually and should not interfere with any other block's entries. For a sequence of length T and head count H, each Softmax call takes as input the vector $\mathbf{s}_i = \{s_{i,0}, \ldots, s_{i,T-1}\}$, for each head $i \in \{0, \ldots, H-1\}$.

We pack all heads into one CKKS vector by:

$$\mathbf{s} = \{ \underbrace{[s_{0,0}, \ldots, s_{0,T-1}]}_{\text{head } 0} \| \cdots \| \underbrace{[s_{H-1,0}, \ldots, s_{H-1,T-1}]}_{\text{head } H-1} \}$$

where $\|$ denotes horizontal concatenation.

All subsequent operations are head-wise, i.e., they act identically on each length-T block and never mix elements across different heads. We achieve this by limiting the rotate-and-sum boundaries to the length T.

Key Takeaway: Computing the true $\max$ in HE is depth-wise expensive and using plaintext $\max$ leaks sensitive information; ELLMo's statistical-$\max$ Softmax using $\mu + p\sigma$ stabilizes exponentials securely while covering a high percentage of the distribution. Further, blockwise head merging enables one-pass Softmax evaluation across packed heads

4.2. LayerNorm with Delayed Normalization

For a token vector $x \in \mathbb{R}^d$, LayerNorm (LN) is

$$\mathrm{LN}(x) = \gamma\,\frac{x - \mu(x)}{\sigma(x)} + \beta, \tag{31}$$

where $\mu(x)$ and $\sigma(x)$ are the mean and standard deviation on the hidden dimension and γ and β are learned parameters. In encrypted computations, the output requires a homomorphic division. Let $\mathrm{EvalPoly}\left(\frac{1}{\sigma(x)}\right)$ denote the polynomial approximation for evaluating the reciprocal of $\sigma(x)$. For high precision, EvalPoly requires a high-degree polynomial and Newton-Raphson iterations [17], [52]. However, we observe that LayerNorm is scale-invariant: multiplying all coordinates of the input by any $\alpha > 0$ leaves the normalized term unchanged. Exploiting this, ELLMo transports the per-token scale forward, eliminating the need to perform EvalPoly. We replace it with inexpensive, uniform multiplications, which are applied to downstream affine terms (weights, biases, and the residual), while preserving the output of the subsequent LayerNorm.

Lemma 4.1. *LayerNorm is invariant to positive scalar scaling of its input. In particular, for any vector $x \in \mathbb{R}^d$ and any scalar $\alpha > 0$, we have*

$$\mathrm{LN}(\alpha x) = \mathrm{LN}(x), \tag{32}$$

up to the learned affine parameters (γ, β).

Proof. Let $\mu(x)$ and $\sigma(x)$ denote the mean and standard deviation of x, respectively. LN is defined as

$$\mathrm{LN}(x) = \gamma\,\frac{x - \mu(x)}{\sigma(x)} + \beta. \tag{33}$$

If $y = \alpha x$ with $\alpha > 0$, then

$$\mu(y) = \alpha\mu(x), \quad \sigma(y) = \alpha\sigma(x). \tag{34}$$

Thus,

$$\frac{y - \mu(y)}{\sigma(y)} = \frac{\alpha x - \alpha\mu(x)}{\alpha\sigma(x)} = \frac{x - \mu(x)}{\sigma(x)}. \tag{35}$$

Applying the same (γ, β) gives $\mathrm{LN}(y) = \mathrm{LN}(x)$. □

Recall that the encoder has two LN blocks, each taking two inputs: 1) the output of the preceding block (x) and 2) a residual (res). A linear layer (O) and a GELU activation sit

between the two LN blocks. For simplicity of discussion, we omit the O and GELU layers here and defer the full analysis to Appendix C. See Figure 2 for details on how the LN inputs differ for the two blocks after the multi-head attention and feed-forward network. For clarity, we index each LN block with a subscript i and refer to the corresponding input as $\tilde{x}_i = (x_i + res_i)$. Then, in a standard encoder, the two LN blocks operate as:

$$\mathrm{LN}_2\ (\mathrm{LN}_1(\cdot)) = \mathrm{LN}_2\ (\underbrace{\mathrm{LN}_1(\tilde{x}_1) + res_2}_{\tilde{x}_2}) \tag{36}$$

Recall our observation that any LN_i is scale-invariant: multiplying both x_i and res_i by any $\alpha > 0$ does not change its output. Rather than dividing x_i by a scale factor (here, σ), we can multiply res_i by the same amount. We refer to this method as ELLMo's DelayNorm (DN).

Definition 4.1. Define the DN block as

$$\mathrm{DN}(\tilde{x}) = \gamma\ (\tilde{x} - \mu) + \beta\ \sigma \tag{37}$$

Then, we propose the following to replace LN_1 with DN.

Proposition 4.1. Scale the parameters β and res_2 by σ. Then, replacing LN_1 with DN preserves correctness.

Proof.

$$\mathrm{DN}(\tilde{x}_1) = \gamma\ (\tilde{x}_1 - \mu) + \beta\ \sigma = \sigma\ \mathrm{LN}_1(\tilde{x}_1).$$

Applying the next LN gives

$$\mathrm{LN}_2(\mathrm{DN}(\tilde{x_1}) + \sigma\ res_2) = \mathrm{LN}_2\big(\sigma\ \mathrm{LN}_1(\tilde{x_1}) + \sigma\ res_2\big)$$

Using Lemma 4.1

$$= \mathrm{LN}_2\big(\mathrm{LN}_1(\tilde{x}_1) + res_2\big) = \mathrm{LN}_2\ (\mathrm{LN}_1(\cdot)) \tag{38}$$

which concludes the proof. □

Note that the carried scale factor σ is computed from the input of LN_1 and does not interfere with the normalization applied at the input of LN_2. Multiplying σ with res_i requires an HEMult, and multiplying the affine parameters and bias in the intermediate linear layer requires PtMult. Thus, we replace the expensive $\mathrm{EvalPoly}$ with cheaper operations: one HEMult and three PtMult.

Although developed for transformer LMs, these techniques generalize to other HE workloads. Statistical-$\max$ also applies to the sigmoid in encrypted logistic regression by stabilizing exponentials without explicit $\max$ operations. DelayNorm extends to normalization layers in encrypted convolutional neural networks by appropriately adjusting residuals and biases. Overall, these strategies reduce multiplicative depth and improve numerical stability across diverse HE-based neural networks.

Key Takeaway: ELLMo replaces the $\mathrm{EvalPoly}$ with multiplications by exploiting the scale-invariance. The next LN removes the remaining scale factor, preserving the standard output while reducing depth.

Table 2: Polynomial settings for nonlinear activations in BERT-Tiny. Composite operations are decomposed and approximated per component.

Function	Method	Degree
$e^{x/2^k}$	Chebyshev + k squarings [15]	59
$1/x$	Comp. Chebyshev & Newton–Raphson	27–27
$1/\sqrt{x}$	Chebyshev & Newton–Raphson	59
$\sqrt{x}$	Chebyshev	27
$\frac{1}{2} + \frac{1}{2}\,\mathrm{erf}\,\frac{x}{\sqrt{2}}$	Chebyshev	119
$\tanh$	Chebyshev	200

5. Evaluation

In this section, we present a comprehensive evaluation of ELLMo for privacy-preserving transformer inference. We begin by describing our experimental methodology, including implementation details and parameter configurations. We then present our results, demonstrating the performance improvements achieved through ELLMo's optimized matrix multiplications and polynomial approximations, along with their impact on both accuracy and latency.

5.1. Methodology

We implement ELLMo on the open source FIDESlib library [4] for GPU-accelerated CKKS operations. For 128-bit security [7], we configure our CKKS-RNS scheme with the following parameters: a polynomial ring degree (N) of 2^{16}, a maximum ciphertext modulus ($\log QP$) of 1740, a multiplicative depth ($L = 26$), effective levels remaining after bootstrapping ($L_{eff} = 10$). The scale factor ($\log q$) is set to 55, and the key-switching parameter $dnum$ is 5.

Table 2 summarizes the polynomial approximation settings for all nonlinear activations in BERT-Tiny. We carefully select polynomial degrees to balance precision and computational cost, while respecting the available multiplicative depth budget.

- Softmax: Exponentials are computed via the scaling identity $e^x = (e^{x/s})^s$ using a degree-59 Chebyshev polynomial, followed by k squarings [15]. The reciprocal for normalization employs two composite Chebyshev approximations (each of degree 27), refined by three Newton–Raphson iterations.
- DelayNorm: We compute μ and σ conventionally and approximate $\sqrt{\sigma}$ using a degree-13 Chebyshev polynomial.
- LayerNorm: We compute μ and σ conventionally and approximate $1/\sqrt{\sigma}$ using a degree-59 Chebyshev polynomial, followed by two Newton–Raphson iterations.
- GELU: We approximate $(1 + \mathrm{erf}(x/\sqrt{2}))/2$ using a degree-119 Chebyshev polynomial, followed by multiplication by x.
- $\tanh$: Approximated with a degree-200 Chebyshev polynomial.

Table 3: **PCMM rotation count comparison with prior work:** For linear layers Q, K, V, O, D, and G, we present the rotation counts to compare ELLMo's shallow PCMM with prior work. For a fair comparison, we analyze two cases sets. The results for case 2 are taken from original papers with parameters $H = 12,\ d = 768,\ d_h = 64, n = 2^{15}$; whereas for case 1 we compute for: $H = 2,\ d = 128,\ d_h = 64, n = 2^{15}$. Here, PF = Powerformer [49] and Flib = FIDESlib [4] and THOR [47].

Case	Layer	PF [49]	Flib [4]	THOR [47]	ELLMo
[1]	$Q/K/V$	393	60	76	30
	O	48	46	32	23
	D/G	192	184	128	92
[2]	$Q/K/V$	2358	549	460	279
	O	966	138	122	93
	D/G	3864	552	488	392

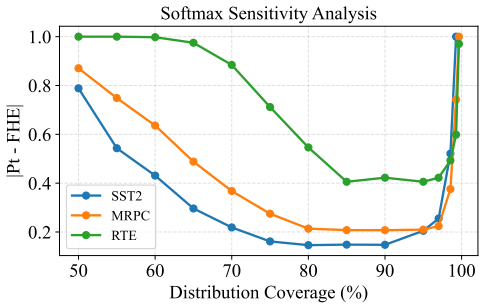
All Chebyshev approximations require inputs normalized to the interval $[-1, 1]$. This scaling operation is woven into the preceding matrix multiplication to save one level.

5.2. Results

5.2.1. Rotation Count Comparison. Table 3 compares the rotation counts for ELLMo's shallow PCMM against prior work across different encoder layers (Q, K, V, O, D, G) for both BERT-Tiny and BERT-Base parameter configurations with $H = 2,\ d = 128,\ d_h = 64, n = 2^{15}$ and $H = 12,\ d = 768,\ d_h = 64, n = 2^{15}$, respectively. We outperform all prior work: Powerformer [49], FIDESlib [4], and THOR [47], across all layer types by a minimum of 28%, and a maximum of 50% reduction in the rotation calls. The fact that relative improvements are maintained as model size increases from BERT-Tiny to BERT-Base parameters confirms that ELLMo's optimization strategy scales effectively to larger architectures.

5.2.2. Statistical $\max$ Estimation Sensitivity Analysis. To analyze how the choice of p in $\max \approx \mu + p\ \sigma$ affects model performance, we evaluate results in terms of logit distance, which measures the model's confidence in binary classification. We report the discrepancy between the logits distance of the encrypted model and the plaintext model (averaged over the dataset) to measure how well encrypted ELLMo performs. Figure 5 presents this analysis.

The results reveal a consistent U-shaped relationship between distribution coverage and logit discrepancy across all tasks. Most critically, attempting to capture more than 95% of the distribution ($p > 2.5$) results in sharp confidence degradation, while the range of $p \in [1, 2]$ (corresponding to 80-95% coverage) maintains minimal and stable logit distance. SST-2 demonstrates particular robustness, maintaining stable accuracy across a wider range of p values, while MRPC exhibits sharper sensitivity outside the optimal range. These findings validate ELLMo's statistical-max approach: targeting 80-95% distribution coverage provides an effective balance between numerical stability and approximation accuracy.



p	%
0	50
0.07	55
0.16	60
0.25	65
0.36	70
0.49	75
0.65	80
0.85	85
1.14	90
1.63	95
2	97
2.50	99
3	99

Figure 5: **Softmax sensitivity vs. distribution coverage.** Logits discrepancy $|\mathrm{HE} - \mathrm{Pt}|$ for SST-2, MRPC, and RTE as a function of distribution coverage (%), with each coverage level mapped to a corresponding p in $\max \approx \mu + p\sigma$ under a heavy-tailed model. All tasks exhibit small discrepancy across roughly 85–95% coverage, while the distance curves become U-shaped with larger mismatch at the extreme tails.

5.2.3. Downstream Task Performance. We perform ablation studies for ELLMo's optimizations: For linear layers, shallow PCMM and masked CCMM are together denoted with efficient transformer (ET); for nonlinear layers, we denote statistical-max Softmax as SS and DelayNorm as DN. 1) Plaintext BERT-Tiny and BERT-Base, and 2) encrypted BERT-Tiny inference and benchmarking on three GLUE downstream tasks: Stanford Sentiment Treebank-2 (SST-2) [48], Microsoft Research Paraphrase Corpus (MRPC) [19], and Recognizing Textual Entailment (RTE) [27]. We report accuracy for SST-2 and RTE, and F1 score for MRPC, following the task-specific evaluation metrics recommended in the GLUE benchmark [66].

Plaintext Results. To assess how well SS and DN scale to larger models, we evaluate BERT-Tiny and BERT-Base in the plaintext domain (Table 4). The packing-aware aspect of transformer is specific to ciphertext computations and hence cannot be evaluated on plaintext models. Therefore, we assess SS and DN in plaintext models. The results demonstrate preservation of model performance at scale: SST-2 accuracy drops by only 0.10%, MRPC F1 decreases by 0.004, and RTE accuracy remains unchanged. These minimal variations provide strong evidence that ELLMo's proposed techniques scale effectively beyond small models.

Encrypted BERT-Tiny Results. Figure 6 presents a comprehensive ablation study evaluating ELLMo's techniques in the encrypted domain. The results demonstrate that ELLMo preserves accuracy under encryption while substantially reducing computational cost. On SST-2, the encrypted model maintains alignment with plaintext performance at ∼83.1% across all optimization stages. Interestingly, SS introduces a minor 0.34% drop, the addition of DN recovers this loss, returning accuracy to baseline levels. This suggests that the approximation errors from different components are capable of eliminating each other in a full inference pass.

5.2.4. Latency Comparison. Table 5 presents an end-to-end latency comparison with the encrypted BERT-Tiny designs. We benchmark against Tricycle [44] (state-of-the-art

Table 4: **Plaintext GLUE results for BERT-Tiny and BERT-Base.** We report SST-2 and RTE accuracy and MRPC F1, following GLUE [66] recommendations. *Base* uses standard LayerNorm and Softmax; *DN* replaces LayerNorm with DelayNorm. FHE-specific PAT is not applicable in plaintext.

Task	BERT-Tiny			BERT-Base		
	Base	SS	SS+DN	Base	SS	SS+DN
SST-2 (Accuracy, %)	83.01	82.89	82.89	92.42	92.32	92.32
MRPC (F1 Score)	0.830	0.822	0.822	0.904	0.900	0.900
RTE (Accuracy, %)	63.18	62.45	62.45	67.15	67.15	67.15

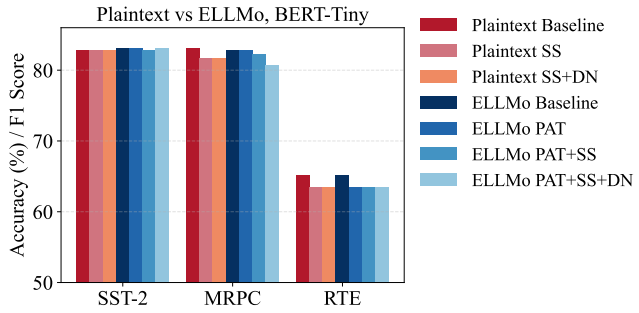


Figure 6: **Plaintext vs. ELLMo, BERT-Tiny.** Grouped bar chart of SST-2 accuracy, MRPC F1, and RTE accuracy for plaintext (red) and ELLMo (blue). Plaintext reports the baseline and DN-SS, while ELLMo includes the baseline and ablations with PAT (packing-aware transformer), SS (statistical-max Softmax), and DN. Metrics follow GLUE recommendations: accuracy for SST-2 and RTE, and F1 for MRPC.

CPU implementation) and FIDESlib [4] (the only available open-source GPU implementation of BERT-Tiny). We note that we could not do a fair direct head-to-head comparison with all prior works due to the following reasons: Powerformer [49] and Encryption-Friendly LLM [54] significantly modify the transformer architecture; and NEXUS, THOR and de Castro *et al.* target larger models at different scales [17], [47], [69].

ELLMo achieves $1.4\times$ speedup over FIDESlib via reduced rotations and 46% fewer bootstrapping calls. This is a cumulative effect of ELLMo's optimizations: PAT reduces rotations and depth, SS and DN replace expensive homomorphic operations with compute-friendly surrogates. The accuracy is maintained within 1% of baseline for most tasks, confirming that ELLMo successfully balances the accuracy-efficiency trade-off.

Table 5: Latency comparison (in seconds) of encrypted BERT-Tiny. Tricycle runtime is on CPU, whereas FIDESlib and ELLMo are on H200 GPUs. The speedup is computed against FIDESlib.

Model	Tricycle [44]	FIDESlib [4]	ELLMo	Speedup
BERT-Tiny	322.0	11.63	8.19	1.42×

6. Related Work

6.1. Privacy-Preserving LLM Inference

HE-Based Methods: NEXUS [69] pioneered non-interactive HE building blocks with high-precision polynomial approximations and packing, though efficient re-packing across layers remained unresolved. Powerformer [49] improved scalability via HE-friendly distillation from BERT-Base to BERT-Tiny, while Power-Softmax [70] replaced exponentials with power functions. EFLLM [54] enabled encrypted fine-tuning using Gaussian-kernel Softmax approximations, but was limited to BERT-Tiny. Most recently, THOR [47] achieved the first encrypted BERT-Base inference with optimized diagonal-packing for matrix multiplications, gaining speedups, though without reducing depth consumption. Tricycle [44] introduces tricyclic encodings, a generalized packing scheme that supports depth-optimal batched matrix multiplications for private transformer inference. Castro *et al.* implement a GPU-accelerated CKKS backend with fast bootstrapping and tailored activation-function approximations, demonstrating encrypted GPT-2 inference [17]. Park *et al.* implement Llama-2 with partially encrypted input sequences, providing a trade-off between security and efficiency [50].

6.2. Algorithmic Improvements

Compiler frameworks simplify deploying ML on FHE backends. HEIR [5] introduces a cross-scheme IR for automatic code generation and HECO [64] offers a Python DSL targeting SEAL [59]. Fhelipe [42] adds a NumPy-style interface with packing and bootstrap placement, and DaCapo [13] optimizes bootstrap scheduling in large models. ELASM [43] improves runtime via new rescaling, while CHLOE [6] compiles complex loop structures. REDSec [25] exploits ternary networks for efficient encrypted inference, and Orion [21] converts PyTorch models into FHE implementations. CERIUM [33] provides a multi-GPU framework that integrates a Python-embedded domain-specific language and an optimizing compiler for large-model FHE inference. These frameworks collectively move FHE from manual tuning toward automated, high-performance pipelines for privacy-preserving ML.

6.3. Hardware Acceleration

Prior work on accelerating fully homomorphic encryption (FHE) broadly falls into two dominant categories. The first line of work proposes custom, application-specific architectures designed exclusively for FHE workloads. Early prototypes [57] were relatively modest in scale, but subsequent designs rapidly expanded to hundreds of mm^2 in silicon area [32], [38], [39], [41], [58]. While these bespoke accelerators deliver substantial performance gains, they incur prohibitive non-recurring engineering costs, including verification, fabrication, and deployment expenses that can

easily reach millions of dollars. These costs significantly limit their practicality and scalability.

This reality motivates a second class of approaches that leverage existing commodity hardware to accelerate homomorphic encryption. Riding the momentum of AI acceleration, these efforts primarily target GPUs [16], [22], [24], [33], [35], [37], [60], [61], and to a lesser extent FPGAs [1], [2], [53], [55], exploiting their massive parallelism and mature software ecosystems. Such platforms offer immediate availability, amortized manufacturing costs, and robust toolchains, making them far more accessible for real-world deployment. In addition, few works also explore heterogeneous acceleration strategies, incorporating in-memory or near-memory computing techniques to mitigate the data-movement bottlenecks inherent in FHE workloads [29], [40].

ELLMo builds on these advances with HE-friendly transformer optimizations that improve efficiency and scalability while preserving accuracy.

7. Conclusion

In this work, we present ELLMo, a system providing efficient algorithmic techniques for privacy-preserving transformer inference under homomorphic encryption. First, ELLMo proposes a shallow PCMM that shifts rotations from ciphertext to plaintext, reducing the number of key-switching calls and levels consumed. Second, ELLMo fuses the masking into CCMM to integrate head-splitting into the linear transforms. Finally, ELLMo provides secure polynomial approximations, including a statistical-$\max$ estimation for Softmax and a delayed normalization for LayerNorm. Implemented on GPU-accelerated FIDESlib and evaluated on BERT-Tiny, ELLMo achieves $1.4\times$ faster runtime with negligible accuracy loss, advancing the practicality of encrypted transformer inference.

References

[1] Rashmi Agrawal, Anantha Chandrakasan, and Ajay Joshi. Heap: A fully homomorphic encryption accelerator with parallelized bootstrapping. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 756–769, 2024.

[2] Rashmi Agrawal, Leo de Castro, Guowei Yang, Chiraag Juvekar, Rabia Yazicigil, Anantha Chandrakasan, Vinod Vaikuntanathan, and Ajay Joshi. Fab: An fpga-based accelerator for bootstrappable fully homomorphic encryption, 2022.

[3] Rashmi Agrawal and Ajay Joshi. *On Architecting Fully Homomorphic Encryption-based Computing Systems*. Springer, 01 2023.

[4] Carlos Agulló-Domingo, Óscar Vera-López, Seyda Guzelhan, Lohit Daksha, Aymane El Jerari, Kaustubh Shivdikar, Rashmi Agrawal, David Kaeli, Ajay Joshi, and José L. Abellán. FIDESlib: A Fully-Fledged Open-Source FHE Library for Efficient CKKS on GPUs. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 1–3, Ghent, Belgium, 2025. IEEE. Poster paper.

[5] Asra Ali, Jaeho Choi, Bryant Gipson, Shruthi Gorantala, Jeremy Kun, Wouter Legiest, Lawrence Lim, Alexander Viand, Meron Zerihun Demissie, and Hongren Zheng. Heir: A universal compiler for homomorphic encryption. *arXiv preprint arXiv:2508.11095*, 2025.

[6] Song Bian, Zian Zhao, Ruiyu Shen, Zhou Zhang, Ran Mao, Dawei Li, Yizhong Liu, Masaki Waga, Kohei Suenaga, Zhenyu Guan, Jiafeng Hua, Yier Jin, and Jianwei Liu. CHLOE: Loop Transformation over Fully Homomorphic Encryption via Multi-Level Vectorization and Control-Path Reduction . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2395–2413, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.

[7] Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R. Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, Xianhui Lu, Carsten Maple, Alberto Pedrouzo-Ulloa, Rachel Player, Yuriy Polyakov, Luis Antonio Ruiz Lopez, Yongsoo Song, and Donggeon Yhee. Security guidelines for implementing homomorphic encryption. *Cryptology ePrint Archive*, 2024.

[8] Stéphane Boucheron, Gábor Lugosi, and Olivier Bousquet. Concentration inequalities. In *Summer school on machine learning*, pages 208–240. Springer, 2003.

[9] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.

[10] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In *Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29-May 3, 2018 Proceedings, Part I 37*, pages 360–384. Springer, 2018.

[11] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A full rns variant of approximate homomorphic encryption. In *Selected Areas in Cryptography–SAC 2018: 25th International Conference, Calgary, AB, Canada, August 15–17, 2018, Revised Selected Papers 25*, pages 347–368. Springer, 2019.

[12] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I 23*, pages 409–437. Springer, 2017.

[13] Seonyoung Cheon, Yongwoo Lee, Dongkwan Kim, Ju Min Lee, Sunchul Jung, Taekyung Kim, Dongyoon Lee, and Hanjun Kim. {DaCapo}: Automatic bootstrapping management for efficient fully homomorphic encryption. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 6993–7010, 2024.

[14] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[15] Wonhee Cho, Guillaume Hanrot, Taeseong Kim, Minje Park, and Damien Stehlé. Fast and accurate homomorphic softmax evaluation. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4391–4404, 2024.

[16] Wonseok Choi, Hyunah Yu, Jongmin Kim, Hyesung Ji, Jaiyoung Park, and Jung Ho Ahn. Theodosian: A deep dive into memory-hierarchy-centric fhe acceleration, 2025.

[17] Leo de Castro, Daniel Escudero, Adya Agrawal, Antigoni Polychroniadou, and Manuela Veloso. Encryptedllm: Privacy-preserving large language model inference via gpu-accelerated fully homomorphic encryption. In *Forty-second International Conference on Machine Learning*, 2025.

[18] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics*, 2019.

[19] Bill Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Third international workshop on paraphrasing (IWP2005)*, 2005.

[20] Léo Ducas and Daniele Micciancio. Fhew: bootstrapping homomorphic encryption in less than a second. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 617–640. Springer, 2015.

[21] Austin Ebel, Karthik Garimella, and Brandon Reagen. Orion: A fully homomorphic encryption framework for deep learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '25, page 734–749, New York, NY, USA, 2025. Association for Computing Machinery.

[22] Guang Fan, Mingzhe Zhang, Fangyu Zheng, Shengyu Fan, Tian Zhou, Xianglong Deng, Wenxu Tang, Liang Kong, Yixuan Song, and Shoumeng Yan. Warpdrive: Gpu-based fully homomorphic encryption acceleration leveraging tensor and cuda cores. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1187–1200, 2025.

[23] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, 2012.

[24] Shengyu Fan, Zhiwei Wang, Weizhi Xu, Rui Hou, Dan Meng, and Mingzhe Zhang. TensorFHE: Achieving Practical Computation on Encrypted Data Using GPGPU . In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 922–934, Los Alamitos, CA, USA, March 2023. IEEE Computer Society.

[25] L. Folkerts, C. Gouert, and N. G. Tsoutsos. Redsec: Running encrypted discretized neural networks in seconds. In *Network and Distributed System Security Symposium (NDSS '23)*, 2023.

[26] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178, 2009.

[27] Danilo Giampiccolo, Bernardo Magnini, Ido Dagan, and William B Dolan. The third pascal recognizing textual entailment challenge. In *Proceedings of the ACL-PASCAL workshop on textual entailment and paraphrasing*, pages 1–9, 2007.

[28] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *International conference on machine learning*, pages 201–210. PMLR, 2016.

[29] Saransh Gupta, Rosario Cammarota, and Tajana Rosing. Memfhe: End-to-end computing with fully homomorphic encryption in memory, 2022.

[30] Shai Halevi and Victor Shoup. Faster homomorphic linear transformations in helib. In *Annual International Cryptology Conference*, pages 93–120. Springer, 2018.

[31] Liam Hodgkinson, Zhichao Wang, and Michael W Mahoney. Models of heavy-tailed mechanistic universality. In *International conference on machine learning*. PMLR, 2025.

[32] Siddharth Jayashankar, Edward Chen, Tom Tang, Wenting Zheng, and Dimitrios Skarlatos. Cinnamon: A framework for scale-out encrypted ai. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS '25, page 133–150, New York, NY, USA, 2025. Association for Computing Machinery.

[33] Siddharth Jayashankar, Joshua Kim, Michael B. Sullivan, Wenting Zheng, and Dimitrios Skarlatos. A scalable multi-gpu framework for encrypted large-model inference. *arXiv preprint 2512.11269*, 2025.

[34] Xiaoqian Jiang, Miran Kim, Kristin Lauter, and Yongsoo Song. Secure outsourced matrix computation and application to neural networks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, page 1209–1222. Association for Computing Machinery, 2018.

[35] Dian Jiao, Xianglong Deng, Zhiwei Wang, Shengyu Fan, Yi Chen, Dan Meng, Rui Hou, and Mingzhe Zhang. Neo: Towards efficient fully homomorphic encryption acceleration using tensor core. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ISCA '25, page 107–121, New York, NY, USA, 2025. Association for Computing Machinery.

[36] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. Tinybert: Distilling bert for natural language understanding. *arXiv preprint arXiv:1909.10351*, 2019.

[37] Jongmin Kim, Wonseok Choi, and Jung Ho Ahn. Cheddar: A swift fully homomorphic encryption library for cuda gpus. *arXiv preprint arXiv:2407.13055*, 2024.

[38] Jongmin Kim, Sangpyo Kim, Jaewan Choi, Jaiyoung Park, Donghwan Kim, and Jung Ho Ahn. Sharp: A short-word hierarchical accelerator for robust and practical fully homomorphic encryption. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–15, 2023.

[39] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1237–1254, 2022.

[40] Jongmin Kim, Sungmin Yun, Hyesung Ji, Wonseok Choi, Sangpyo Kim, and Jung Ho Ahn. Anaheim: Architecture and algorithms for processing fully homomorphic encryption in memory. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1158–1173, 2025.

[41] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, John Kim, Minsoo Rhu, and Jung Ho Ahn. Bts: an accelerator for bootstrappable fully homomorphic encryption. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ISCA '22, page 711–725, New York, NY, USA, 2022. Association for Computing Machinery.

[42] Aleksandar Krastev, Nikola Samardzic, Simon Langowski, Srinivas Devadas, and Daniel Sanchez. A tensor compiler with automatic data packing for simple and efficient fully homomorphic encryption. *Proc. ACM Program. Lang.*, 8(PLDI), June 2024.

[43] Yongwoo Lee, Seonyoung Cheon, Dongkwan Kim, Dongyoon Lee, and Hanjun Kim. ELASM: Error-latency-aware scale management for fully homomorphic encryption. In *32nd USENIX Security Symposium (USENIX Security '23)*, pages 4697–4714. USENIX Association, 2023.

[44] Lawrence Lim, Vikas Kalagi, Divyakant Agrawal, and Amr El Abbadi. Tricycle: Private transformer inference with tricyclic encodings. Cryptology ePrint Archive, Paper 2025/1200, 2025.

[45] Zeyu Liu, Daniele Micciancio, and Yuriy Polyakov. Large-precision homomorphic sign evaluation using fhew/tfhe bootstrapping. In *Advances in Cryptology – ASIACRYPT 2022: 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5–9, 2022, Proceedings, Part II*, page 130–160, Berlin, Heidelberg, 2022. Springer-Verlag.

[46] John C Mason and David C Handscomb. *Chebyshev Polynomials*. Chapman and Hall/CRC, Boca Raton, FL, 2002.

[47] Jungho Moon, Dongwoo Yoo, Xiaoqian Jiang, and Miran Kim. THOR: Secure transformer inference with homomorphic encryption, 2025.

[48] Bo Pang and Lillian Lee. Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. *Proceedings of the 43rd Annual Meeting on Association for Computational Linguistics (ACL)*, 2005.

[49] Dongjin Park, Eunsang Lee, and Joon-Woo Lee. Powerformer: Efficient and high-accuracy privacy-preserving language model with homomorphic encryption. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11090–11111, Vienna, Austria, July 2025. Association for Computational Linguistics.

[50] Jaiyoung Park, Sejin Park, Jai Hyun Park, Jung Ho Ahn, Jung Hee Cheon, Guillaume Hanrot, Jung Woo Kim, Minje Park, and Damien Stehlé. Scaling up privacy-preserving ml: A ckks implementation of llama-2-7b, 2026.

[51] Michael S Paterson and Larry J Stockmeyer. On the number of nonscalar multiplications necessary to evaluate polynomials. *SIAM Journal on Computing*, 2(1):60–66, 1973.

[52] Hongyuan Qu and Guangwu Xu. Improvements of homomorphic secure evaluation of inverse square root. In *Information and Communications Security*, pages 110–127, Singapore, 2023. Springer Nature Singapore.

[53] Xuanle Ren, Zhaohui Chen, Zhen Gu, Yanheng Lu, Ruiguang Zhong, Wen-Jie Lu, Jiansong Zhang, Yichi Zhang, Hanghang Wu, Xiaofu Zheng, Heng Liu, Tingqiang Chu, Cheng Hong, Changzheng Wei, Dimin Niu, and Yuan Xie. Cham: A customized homomorphic encryption accelerator for fast matrix-vector product. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2023.

[54] Donghwan Rho, Taeseong Kim, Minje Park, Jung Woo Kim, Hyunsik Chae, Ernest K. Ryu, and Jung Hee Cheon. Encryption-friendly llm architecture, 2025.

[55] M. Sadegh Riazi, Kim Laine, Blake Pelton, and Wei Dai. Heax: An architecture for computing on encrypted data. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '20, page 1295–1309, New York, NY, USA, 2020. Association for Computing Machinery.

[56] Lorenzo Rovida and Alberto Leporati. Transformer-based language models and homomorphic encryption: An intersection with bert-tiny. In *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*, pages 3–13, 2024.

[57] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. F1: A fast and programmable accelerator for fully homomorphic encryption. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, page 238–252, New York, NY, USA, 2021. Association for Computing Machinery.

[58] Nikola Samardzic, Axel Feldmann, Aleksandar Krastev, Nathan Manohar, Nicholas Genise, Srinivas Devadas, Karim Eldefrawy, Chris Peikert, and Daniel Sanchez. Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ISCA '22, page 173–187, New York, NY, USA, 2022. Association for Computing Machinery.

[59] SEAL. Microsoft SEAL (release 4.1). https://github.com/Microsoft/SEAL, January 2023. Microsoft Research, Redmond, WA.

[60] Kaustubh Shivdikar, Yuhui Bao, Rashmi Agrawal, Michael Shen, Gilbert Jonatan, Evelio Mora, Alexander Ingare, Neal Livesay, José L. Abellán, John Kim, Ajay Joshi, and David Kaeli. Gme: Gpu-based microarchitectural extensions to accelerate homomorphic encryption. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '23, page 670–684, New York, NY, USA, 2023. Association for Computing Machinery.

[61] Jianming Tong, Tianhao Huang, Jingtian Dang, Leo de Castro, Anirudh Itagi, Anupam Golder, Asra Ali, Jeremy Kun, Jevin Jiang, Arvind, G. Edward Suh, and Tushar Krishna. Leveraging asic ai chips for homomorphic encryption, 2026.

[62] Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Neural Information Processing Systems*, 2017.

[63] Roman Vershynin. *High-Dimensional Probability: An Introduction with Applications in Data Science*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 2 edition, 2026.

[64] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. Heco: Fully homomorphic encryption compiler. In *32nd USENIX Security Symposium (USENIX Security '23)*, pages 4715–4732, 2023.

[65] Mariia Vladimirova, Jakob Verbeek, Pablo Mesejo, and Julyan Arbel. Understanding priors in bayesian neural networks at the unit level. In *International Conference on Machine Learning*, pages 6458–6467. PMLR, 2019.

[66] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP workshop BlackboxNLP: Analyzing and interpreting neural networks for NLP*, pages 353–355, 2018.

[67] Tianshi Xu, Wen-jie Lu, Jiangrui Yu, Yi Chen, Chenqi Lin, Runsheng Wang, and Meng Li. Breaking the layer barrier: Remodeling private transformer inference with hybrid ckks and mpc. *34th USENIX Security Symposium (USENIX Security 25)*, pages 2653–2672, 2025.

[68] Hao Yang, Shiyu Shen, Wangchen Dai, Lu Zhou, Zhe Liu, and Yunlei Zhao. Phantom: A cuda-accelerated word-wise homomorphic encryption library. *IEEE Transactions on Dependable and Secure Computing*, 21(5):4895–4906, 2024.

[69] Jiawen Zhang, Jian Liu, Lipeng He, Xinpeng Yang, Yinghao Wang, Kejia Chen, Xiaoyang Hou, Kui Ren, and Xiaohu Yang. Secure transformer inference made non-interactive. *Network and Distributed System Security (NDSS)*, 2024:136, 2025.

[70] Itamar Zimerman, Allon Adir, Ehud Aharoni, Matan Avitan, Moran Baruch, Nir Drucker, Jenny Lerner, Ramy Masalha, Reut Meiri, and Omri Soceanu. Power-softmax: Towards secure llm inference over encrypted data. *arXiv preprint arXiv:2410.09457*, 2024.

[71] Ali Şah Özcan and Erkay Savaş. HEonGPU: a GPU-based fully homomorphic encryption library 1.0. Cryptology ePrint Archive, Paper 2024/1543, 2024.

8. LLM usage considerations

We used large language models (LLMs) only as writing assistants to improve grammar, wording, and structure of paragraphs and figure captions. All technical ideas, constructions, algorithms, proofs, and experimental designs and results were developed, implemented, and validated by the authors. The literature review and choice of related work to cite were conducted by the authors using standard academic tools, not delegated to LLMs. LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality.

Appendix A. Comparison against Prior Work

We benchmark against Tricycle [44] (state-of-the-art CPU implementation) and FIDESlib [4] (the only available open-source GPU implementation of BERT-Tiny). We also consider Rovida *et al.* [56], which provides an end-to-end BERT-Tiny implementation on CPU; however, since it is slower than Tricycle in this setting, we report Tricycle as our CPU baseline. We could not perform a fair, direct head-to-head comparison with all prior works: Powerformer [49] and EFLLM [54] significantly modify the transformer architecture, NEXUS [69] does not provide a reproducible end-to-end implementation, and THOR [47] and Castro *et al.* [17] target larger models at different scales.

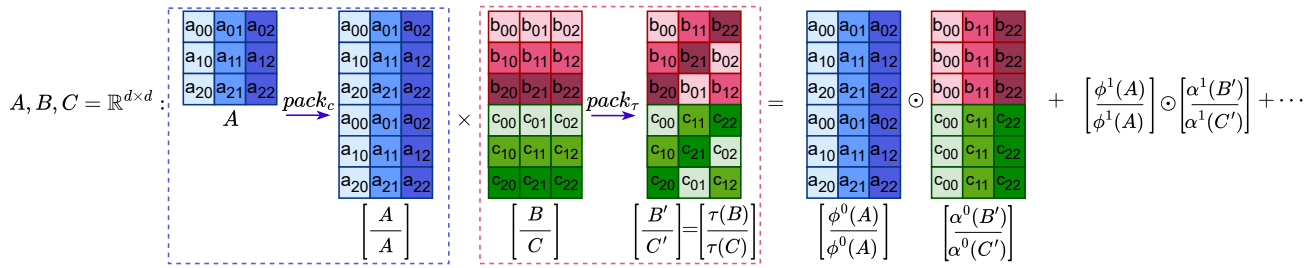


Figure 7: **Multi-PCMM variant for batched parallel multiplications:** When multiple $T \times d$ matrices fit within a single ciphertext, they are vertically concatenated and processed in a single PCMM operation. Because ϕ^k acts only along the column dimension and the plaintext transforms are applied per block before encoding, no slots mix across block boundaries, so the output contains all products vertically concatenated.

Appendix B. Shallow Multi-PCMM

We also propose a multi-PCMM variant for our algorithm. Recall from Section 2 that MHA runs three linear projections Q, K, V in parallel. When the slot budget n permits, we pack $p = \frac{n}{d \cdot T}$ matrices per ciphertext via vertical concatenation.

Let A be a ciphertext matrix and B_k denote the k-th plaintext matrix multiplying A. Pack p matrices as:

$$\begin{aligned} \widetilde{B} &= [B_0 \| \dots \| B_{p-1}] \in \mathbb{R}^{pT \times d_h} \\ \widetilde{A} &= [A \ \| \dots \| A \ ] \in \mathbb{R}^{pT \times d_h} \end{aligned} \tag{39}$$

where $\|$ denotes vertical concatenation. Proposition B.1 computes PCMM $A \times B_k$ for $k \in [p]$ simultaneously.

Proposition B.1. Correctness of Multi-PCMM:

$$[A \times B_0 \| \dots \| A \times B_k] = \sum_{k=0}^{d-1} \left(\phi^k(\widetilde{A})\right) \odot \left(\zeta^k \circ \tau(\widetilde{B})\right) \tag{40}$$

Proof. The linear transform ϕ^k permutes entries within each d-wide row and never across rows ($\phi^k(A)_{i,j} = A_{i,\, j+k}$), so it applies identically and independently to every $T \times d$ block of $\widetilde{A}$. Since $\widetilde{A}$ repeats A in each block, $\phi^k(\widetilde{A})$ equals $\phi^k(A)$ replicated vertically. The plaintext transforms τ and ζ^k are computed per-block before encoding, placing $\zeta^k(\tau(B_j))$ in the slots corresponding to block j. Element-wise multiplication and summation are slotwise, so block j accumulates $\sum_{k=0}^{d-1} \phi^k(A) \odot \zeta^k(\tau(B_j)) = A \times B_j$ by Proposition 13. Since no operation mixes slots across block boundaries, the output ciphertext contains $[A \times B_0 \| \dots \| A \times B_{p-1}]$. □

Appendix C. LayerNorm with Delayed Normalization

We now show that the FFN layers are effectively scale-invariant, so the scale produced by the first LayerNorm (LN1) can be carried through to the second (LN2). Let x be the per-token hidden vector, μ its mean, and $u = x - \mu$ the centered vector. Standard LayerNorm computes the variance $v = \frac{1}{d}\sum_\ell u_\ell^2$ and outputs

$$h = \frac{x - \mu}{\sqrt{v + \varepsilon}}.$$

With DelayNorm, we keep only u and track the inverse standard deviation

$$s = \tfrac{1}{\sigma},$$

so the logical normalized output is $h = s\,u$ without applying s homomorphically.

For the FFN down and gate projections,

$$z_{\text{down}} = W_{\text{down}} h + b_{\text{down}} = s\Big(W_{\text{down}} u + \tfrac{1}{s} b_{\text{down}}\Big),$$
$$z_{\text{gate}} = W_{\text{gate}} h + b_{\text{gate}} = s\Big(W_{\text{gate}} u + \tfrac{1}{s} b_{\text{gate}}\Big).$$

We therefore apply the matrices to u and use scaled biases

$$\tilde{b}_{\text{down}} = \tfrac{1}{s} b_{\text{down}}, \qquad \tilde{b}_{\text{gate}} = \tfrac{1}{s} b_{\text{gate}},$$

so that $z = s(\tilde{z} + \tilde{b})$ holds logically.

The gate uses

$$g = \text{GeLU}(z_{\text{gate}}) = \text{GeLU}\big(s(\tilde{z}_{\text{gate}} + \tilde{b}_{\text{gate}})\big).$$

GeLU is not exactly scale-invariant, but in the LayerNorm regime ($\sigma \approx 1$ and inputs near zero) the effect of s is close to a smooth rescaling that the following linear layer can absorb, letting us avoid the encrypted reciprocal and square root.

C.1. Statistical-max Estimation

Table 6 shows coverage for values of p used in our experiments.

Table 6: Distribution coverage for statistical-max estimation.

p	0	0.08	0.16	0.25	0.36	0.50
% Coverage	50	55	60	65	70	75
p	0.65	0.85	1.14	1.63	2	2.5
% Coverage	80	85	90	95	97	98.54

Setting $p \in [1, 2]$ provides 85–95% coverage, sufficient to prevent overflow while avoiding the cost of exact max computation. This aligns with empirical measurements of attention logit distributions [31], [65].